

TaxaMatch

Store and standardize biological match data for the TaxaID ecosystem. Ingests raw match results from external classification tools and produces a canonical match object for input to TaxaLikely (likelihood conversion) or TaxaAssign (direct assignment).

The main input to TaxaID is a table where each row pairs an observation (sequence, image, or sound recording) with a candidate taxon from a reference database and a match score indicating their similarity. TaxaMatch standardizes column names and taxonomy so that downstream packages can interpret this table. Standardization is a prerequisite for calibrating scores into likelihoods – without it, raw scores may be mistaken for probabilities, leading to overconfident or inconsistent assignments.

TaxaMatch can also produce match tables from raw sequence data via NCBI BLAST, though most users will start from an existing bioinformatics pipeline.

Supported Data Types

Data type	Input format	Function
DNA sequences	DADA2 seqtab, FASTA, DNASTringSet	<code>read_sequence_table()</code>
BLAST results	Remote NCBI or local rBLAST	<code>blast_sequences()</code>
Images	Animl CSV export	<code>read_animl_output()</code>
Acoustics	BirdNET-Analyzer CSV	<code>read_birdnet_output()</code>

Installation

```
# Requires TaxaTools (foundation package)
devtools::install("path/to/TaxaTools")
devtools::install("path/to/TaxaMatch")
```

Quick Start

```
library(TaxaMatch)

# 1. Read DADA2 output
seqs <- read_sequence_table("seqtab_nochim.rds")

# 2. Filter by length and abundance
filtered <- filter_sequences(seqs, min_length = 100, max_length = 300,
                             min_reads = 10)

# 3. BLAST against NCBI
blast_hits <- blast_sequences(filtered, database = "nt",
                              barcode_term = "12S",
                              min_score = 80)

# 4. Standardize to canonical match object
match_df <- standardize_match_data(blast_hits)
```

```
# 5. Remove redundant higher-rank hypotheses
match_df <- filter_redundant_hypotheses(match_df)
# Result: one row per observation_id x taxon hypothesis, ready for TaxaLikely
```

Key Functions

- `read_sequence_table()` – ingest DADA2 seqtab matrix, FASTA, or DNASTringSet
- `filter_sequences()` – filter ASVs by length range and minimum abundance
- `blast_sequences()` – remote NCBI BLAST or local rBLAST with score window filtering and taxonomy resolution
- `read_birdnet_output()` – ingest BirdNET-Analyzer CSV files; `observation_id` encodes recording + time window ("`{stem}_{start_s}-{end_s}`"); `score` is BirdNET confidence (0–1)
- `read_animl_output()` – ingest Animl (MegaDetector + SpeciesNet) camera trap CSV; `observation_id` = image filename stem; configurable column names; supports long and wide (`n_candidates`) formats
- `standardize_match_data()` – canonical column names, taxonomy derivation via `TaxaTools::create_taxon_names()`
- `filter_redundant_hypotheses()` – drop coarser-rank rows superseded by finer-rank rows within the same lineage and observation
- `report_match()` – summarize matching for `assemble_report()`

Acoustic Workflow

BirdNET-Analyzer is a free Python tool from the Cornell Lab of Ornithology that classifies bird vocalizations in audio files.

Install BirdNET-Analyzer (requires Python 3.9+, ~100 MB model download):

```
pip3 install birdnetlib
```

Analyze audio files and read results into R:

```
library(TaxaMatch)

# Run BirdNET-Analyzer on a directory of audio files
script <- '
from birdnetlib import Recording
from birdnetlib.analyzer import Analyzer
import os, csv

os.makedirs("birdnet_results", exist_ok=True)
analyzer = Analyzer()
for fname in os.listdir("reference_audio/"):
    if not fname.endswith(".mp3"):
        continue
    rec = Recording(analyzer, os.path.join("reference_audio/", fname),
                    lat=37.5, lon=-122.0, min_conf=0.1)
    rec.analyze()
    out = os.path.join("birdnet_results", fname.replace(".mp3", ".BirdNET.results.csv"))
    with open(out, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["Start (s)", "End (s)", "Scientific name", "Common name", "Confidence"])
        for d in rec.detections:
```

```

        w.writerow([d["start_time"], d["end_time"],
                     d["scientific_name"], d["common_name"], d["confidence"]])
    ,
writeLines(script, "/tmp/run_birdnet.py")
system("python3 /tmp/run_birdnet.py")

# Read results into match object format
match_df <- read_birdnet_output("birdnet_results/",
                               min_confidence = 0.1,
                               top_n          = 3L)

# Standardize and pass to TaxaLikely
match_df <- standardize_match_data(match_df,
                                   observation_id_col = "observation_id",
                                   score_col         = "score")

```

Expected BirdNET output format: one CSV per audio file, named `recording.BirdNET.results.csv`, with columns `Start (s)`, `End (s)`, `Scientific name`, `Common name`, `Confidence`. The BirdNET-Analyzer default produces up to 3 detections per 3-second window. Use `top_n = 1` to keep only the best candidate per window.

Reference training workflow: Download ground-truth recordings from Xeno-canto with `TaxaLikely::fetch_reference_`, run BirdNET-Analyzer on the downloaded audio, then join detections back to the known species via `source_file` to label H1/H2/H3 training examples.

Camera Trap Image Workflow

Animl (MegaDetector + SpeciesNet) is an R package on CRAN that classifies camera trap images. Install it with `install.packages("animl")` (requires Python 3.12 via `reticulate`).

Read Animl results into match object format:

```

library(TaxaMatch)

# Long-format Animl CSV (one row per image x candidate species)
match_df <- read_animl_output(
  "animl_results/",          # directory of Animl export CSVs
  min_confidence = 0.5       # drop low-confidence detections
) |>
  # Remove non-wildlife detections before standardizing
  subset(!species %in% c("empty", "human", "vehicle"))

# Wide-format output (pred1/score1, pred2/score2, pred3/score3 per row)
match_df_wide <- read_animl_output(
  "animl_results/wide.csv",
  species_col = "pred",
  score_col   = "score",
  n_candidates = 3L
)

# Standardize and pass to TaxaLikely
match_df <- standardize_match_data(match_df,
                                   observation_id_col = "observation_id",
                                   score_col         = "score")

```

Expected Animl output format (long): one or more CSV files with columns `FileName`, `prediction`, `confidence`. `FileName` is the image path; `prediction` is the species label (scientific name, "empty", "human", or "vehicle"); `confidence` is the classifier posterior (0–1). Column names are configurable via `file_col`, `species_col`, and `score_col`.

Coverage audit: Check which expected species are absent from the classifier’s known list with `TaxaLikely::audit_acoustic_coverage(plausible_species, reference_species)`. Pass the result’s `$unreferenced` vector to `TaxaAssign::suggest_unreferenced_species()` as `unreferenced_taxa`.

Coverage column: `read_animl_output()` can retain the bounding box area (width × height, normalised 0–1) as a `coverage` column when `bbox_cols` are specified. Smaller bounding boxes indicate a partially visible or distant animal — lower evidence per detection, directly analogous to DNA alignment coverage and acoustic recording quality. Use `TaxaLikely::coverage_threshold()` or `TaxaLikely::calibrate_coverage_filter()` to choose a filter cutoff before model training.

Other Image Classifiers

The match object format is classifier-agnostic. Three dedicated reader functions are available; any other tool that returns a species label and a confidence score per image can be adapted manually.

iNaturalist computer vision returns a ranked list of up to 10 candidate taxa (species, genus, or family) with softmax confidence scores (0–1) via the public API (<https://api.inaturalist.org/v2/>). Its 108,000+ taxon training set covers general wildlife globally. Use `read_inaturalist_cv_output()` on saved JSON response files:

```
# Save API response as JSON (one file per image), then:
inat_df <- read_inaturalist_cv_output(
  "inat_results/",          # directory of per-image JSON files
  score_type = "combined_score",
  min_confidence = 0.05,
  top_n = 5L
) |> subset(taxon_rank == "species")
```

Wildlife Insights / SpeciesNet (Google/WCS) processes camera trap images with the open-source SpeciesNet Python model. Use `read_wildlife_insights_output()` on the batch predictions JSON:

```
# Run SpeciesNet: python -m speciesnet.scripts.run_model \
# --folders images/ --predictions_json speciesnet_output.json
wi_df <- read_wildlife_insights_output(
  "speciesnet_output.json",
  min_confidence = 0.3
) |> subset(!species %in% c("blank", "human", "vehicle"))
```

InsectNet (He et al. 2025; <https://insectapp.las.iastate.edu>) targets insects (2,526 species, 17 orders) with 96.4% top-1 accuracy. Unlike the classifiers above, it returns *conformal prediction sets* rather than a single ranked-confidence list — a set of species guaranteed to contain the true species with 97.5% probability. It also flags out-of-distribution images with an energy-based OOD score. The conformal output format is not currently compatible with the score-based likelihood model in `TaxaLikely` without access to the underlying softmax scores. A web interface is available; programmatic access to model weights is described in the paper but no public API exists at time of writing.

Classifier	Reader function	Score type	R package
Animl / SpeciesNet	<code>read_animl_output()</code>	Confidence 0–1	animl (CRAN)
iNaturalist CV	<code>read_inaturalist_cv_output()</code>	Softmax 0–1	rinat (indirect)
Wildlife Insights / SpeciesNet	<code>read_wildlife_insights_output()</code>	Confidence 0–1	Python speciesnet
InsectNet	<i>(not yet compatible)</i>	Conformal sets	Web app only

Match Object Output

The canonical match object has one row per `observation_id` x reference match:

Column	Description
<code>observation_id</code>	Query identifier (ESV/ASV ID)
<code>score_original</code>	Raw match score (original scale, never modified)
<code>taxon_name</code>	Best taxon label at finest available rank
<code>taxon_name_rank</code>	Rank of <code>taxon_name</code> (species, genus, etc.)
<code>family, genus, species, ...</code>	Taxonomy columns

Vignettes

- Match Standardization – full workflow

Part of TaxaID

TaxaMatch standardizes match data for two downstream paths: TaxaLikely (Bayesian likelihood model) or TaxaAssign (direct LLM-based assignment).

Ecosystem: TaxaTools -> **TaxaMatch** -> TaxaLikely -> TaxaAssign

See the TaxaID README for ecosystem overview and installation instructions.

Citation

Lafferty, K.D., 2026, TaxaID – A modular R ecosystem for Bayesian taxonomic assignment: U.S. Geological Survey software release, <https://doi.org/10.5066/xxxxxx>.

Software Requirements

- R ($\geq 4.1.0$)
- TaxaTools (foundation package, installed first)
- httr2 and rentrez (for remote NCBI BLAST)
- Biostrings and rBLAST (optional; for local BLAST)
- Python 3.9+ and birdnetlib (`pip3 install birdnetlib`; optional, for acoustic analysis via BirdNET-Analyzer)

All dependencies are declared in the DESCRIPTION file and installed automatically.

Developed with Claude Code (Anthropic).